Start here: what you are learning

Agksheya B. | Reporting, Administration and Customer Support
Personal preparation pack, based on the local project inspected on 8 October 2026.

This guide starts from the basics. Read an explanation, open the named file, find the operation and explain it aloud. Understanding comes from connecting a screen action to its code and saved data. Reading once cannot guarantee that you can answer every examiner question; the practice checks help you discover what needs another look.

Your opening explanation

LaundryLink coordinates a laundry business. A customer places an order, payment is reviewed, a rider collects the clothes, staff process and check them, and a rider delivers them. Your component adds customer support, administration and reports to that shared system.

Say it in your own words: My component receives enquiries, complaints and feedback. It lets CSM coordinate cases, lets the assigned person reply, and lets customers confirm whether they need more help. It also manages catalogue and operational accounts and generates reports from stored records.

Read	Purpose
2-6	Understand the application, Java basics, layers, APIs and database.
7-13	Understand your support workflow and its safeguards.
14-17	Understand administration, reporting, notifications and page ownership.
18-22	Understand OOP and the two pattern examples.
23-28	Locate code, rehearse the demo and test your understanding.

This is a personal learning document, not the required group final report. Code excerpts are shortened for readability; use the source locator at the end to find the actual implementation. The project itself is not changed by this guide.

The whole system and your responsibility

Module	Purpose
Accounts	Registration, login, profile and saved addresses.
Orders	Selected services/items, booking, order details and modifications.
Processing	Receipt, cleaning routes, quality checks, rework and packing.
Pickup and delivery	Assigned rider tasks and transport outcomes.
Finance	Payment recording/verification, billing and promotions.
Your component	Reporting, administration and customer support.

Why modules share an order ID

An order ID identifies one booking. The order lines describe its items, payments refer to that order, delivery refers to the same order, and a complaint may refer to it too. Sharing the ID connects the work without copying unrelated records. A case ID identifies a separate support request: case #70 could concern order #197; the numbers need not match.

Your boundary

Your main Java package is support/. It contains SupportController, SupportService, SupportRepository, SupportAccess, ReportService, AdministrationService, DTOs and rider-assignment coordination. Shared notifications and security support your code. Processing and promotion Strategy implementations belong to other modules: explain their integration, but do not claim them as your own exclusive implementation.

Three systems in the local setup

The browser displays pages and runs JavaScript. The Spring Boot application runs Java and accepts requests at localhost:8080. SQL Server stores the persistent records. localhost means this computer; 8080 is the application port. SQL Server is the database server, not the HTML page or Java service class.

Check yourself: Does closing a support case complete a laundry order?

Answer: No. The case and order have separate identifiers, records and status fields.

Reading Java without getting lost

```
private final SupportRepository repo;

public SupportService(SupportRepository repo) {
    this.repo = repo;
}
```

Read the first line

private restricts direct access to the field. final prevents that reference being reassigned after initialization. SupportRepository is the type of collaborator. repo is its variable name. The field lets the service use a repository object. final does not make the repository or database immutable.

Read the constructor

A constructor initializes an object and has the same name as its class. The parameter repo is the incoming dependency. this.repo means the field of the current object. Spring supplies the dependency in the real constructor, which also receives access and notification collaborators.

```
public Map<String, Object> create(Actor actor, CaseInput input)
```

public means callers can invoke the method. Map<String,Object> is its return type: named keys map to values. create is the method name. Actor and CaseInput are parameter types; actor and input are the parameter names. A return passes a result back. A throw stops the normal flow with an error.

Common syntax

if checks a condition. ! means not. && means both conditions must hold; || means either. null means no value. List is a collection; Set is used for unique values such as allowed roles. An Integer can be null, unlike primitive int. A record is a compact Java data-carrier type with generated accessors such as input.type().

Check yourself: Is actor.staff() the STAFF job role only?

Answer: No. In SupportAccess it means any permitted operational role, including manager, owner, CSM, staff and rider.

Spring Boot layers: what each does

Term	Meaning and your example
Controller	HTTP entry point. SupportController binds URL/input and delegates. It does not perform the whole workflow itself.
Service	Business rules and orchestration. SupportService checks permissions, statuses and feedback rules before writing.
Repository	Database operations. SupportRepository wraps JdbcTemplate. SQL is still often assembled in services.
DTO	Data Transfer Object. SupportRequests describes the expected payload and constraints. It is not automatically a table.
Entity	Java object mapped to a database table using JPA, such as User or Order. Your support module mostly returns JDBC maps.
Bean	Object created/managed by Spring. @Service and @Repository help register those objects.
Configuration	Application settings and bean/security setup, including SecurityConfig.
Exception advice	Converts errors into useful HTTP responses, for example SupportErrors.

The request journey

HTML button -> JavaScript handler -> HTTP request -> security/session checks -> controller -> service -> repository/database -> response -> JavaScript redraws the screen. These layers are responsibilities, not necessarily separate computers.

Do not confuse the word service

services/Service.java represents a laundry catalogue service, such as Wash and Fold. ServiceService is business logic for that feature. SupportService is a business service class. None of those classes is the whole web server.

Say it in your own words: The controller handles the request, the service decides what is permitted, and database operations persist the result.

APIs, JSON, annotations and errors

```
POST /api/support/cases
{
  "type": "Question",
  "subject": "Explain my charge",
  "message": "How was this amount calculated?",
  "orderId": 197,
  "topic": "Payments & billing"
}
```

This is an illustrative customer submission. JSON names carry values from JavaScript to Java. The customer identity comes from the authenticated actor; it should not be trusted from a user-entered customer ID. General enquiries can omit an order where the workflow permits it.

Annotation	Meaning
@RestController	Expose HTTP endpoints that return response data.
@RequestMapping	Common URL prefix: /api/support.
@GetMapping / @PostMapping	Read a resource / submit an operation.
@PutMapping / @PatchMapping	Replace/edit case input / update handling fields.
@RequestBody / @PathVariable	Bind JSON input / extract {id} from a URL.
@Valid	Check DTO constraints before normal controller processing.
@Transactional	Commit related database changes together or roll them back.

Useful HTTP responses

201 means created; 204 means success with no response body; 400 means invalid input; 403 means forbidden; 404 means unavailable resource; 409 means conflicting state or stale update. Check the actual handler because individual endpoints can differ. JavaScript should display the error instead of pretending that a save succeeded.

Check yourself: Why is hiding a button insufficient?

Answer: A caller can send the API request directly. Backend permission checks must reject forbidden operations.

The database: records, keys and relationships

Table	What it stores
users	Customer/operational identity, email, encoded password and role.
orders / orderLines	A booking and its chosen items, quantities, services and line totals.
feedback	Both support requests and one-time feedback; type distinguishes journeys.
chat	Messages linked to a feedback/case record and their author.
support_activity	Audit entries describing actions and actors.
notifications	Recipient messages, links and read state.
services / servicePricing	Catalogue definitions and item/service prices.
payments / delivery / logs	Payment records, transport records and order status history.

Primary key and foreign key

A primary key uniquely identifies a row: feedbackID identifies a case. A foreign key connects related rows: chat.feedbackID points to its case, and feedback.orderID can point to a related order. A join reads connected data, such as the customer name next to the case. LEFT JOIN permits missing optional data, such as an unassigned case.

What persistence means

An INSERT creates a row, SELECT reads rows, UPDATE changes a row and DELETE removes one. Soft deletion sets a flag so normal reads exclude the record while retaining history. Refreshing a page reads persisted data again; it should not erase a saved request.

Two different status fields

orders.statusID describes physical laundry progress. feedback.caseStatus describes support progress. A complaint about a Delivered order can still be New. Resolving that complaint should not rewrite its laundry history.

Check yourself: Why are replies not stored in support_activity?

Answer: chat contains customer-visible conversation; support_activity records an audit trail. They serve different purposes.

Submitting a case: follow the actual code

Where to open

Customer page: html/customer/feedback.html. Browser controller: js/support/cases.js. API: POST /api/support/cases. Java: SupportController.create -> SupportService.create -> SupportRepository.

Step 1: field validation

CaseInput permits Complaint, Feedback or Question. Subject is nonblank and at most 100 characters; original message is nonblank and at most 500. Order ID is positive if supplied. Topics are restricted to known values. Feedback requires a rating from 1 to 5; questions and complaints must not carry a rating.

Step 2: actor and order validation

SupportService.validateCase requires a customer actor. If an order is supplied, it counts orders matching both that order ID and actor ID. An existing order belonging to another customer is not acceptable. This is business/ownership validation beyond DTO field checks.

```
INSERT INTO feedback(
    feedback, userID, orderID, caseType, subject, rating, topic
)
... VALUES (?, ?, ?, ?, ?, ?, ?);
```

The message goes into the feedback column, actor.id() into userID and the selected order into orderID. The insert returns the generated case ID. SQL Server OUTPUT is used to retrieve the new identifier. The initial case fields also rely on database defaults.

Step 3: audit, notification and response

The service adds a Created audit entry and notifies CSM roles. Feedback links to the feedback workspace; enquiries/complaints link to cases. It returns detail(actor,id), so the browser can show the saved result. @Transactional groups the database writes.

Say it in your own words: Creation validates the customer and order ownership, inserts the request, records the action and notifies CSM.

Reading, searching, editing and deleting

Reading is permission-scoped

SupportService.cases returns a filtered/paginated list. Customers see their own records. CSM and owner/admin oversight can see all cases. Other operational users see assigned records. Detail reads use one(actor,id), which checks access rather than trusting the ID in the browser.

Filters and pagination

The service builds SQL conditions for search, status, type, priority, assignee and topic. Pagination requests a page of results instead of returning every row. Default case lists exclude Feedback, which has its own view. Empty results mean no matching records; they do not necessarily mean the database is disconnected.

Customer editing

Edit uses PUT /cases/{id} with CaseInput. Eligibility depends on customer ownership, New status and the version. One-time feedback cannot be treated as an editable ongoing chat. Look at SupportService.edit for the exact restrictions before claiming an operation is allowed.

Customer deletion

Delete uses DELETE /cases/{id}?version=... and SupportService.delete. The case is soft-deleted rather than destroying its history. Customers cannot remove someone else's record or freely remove a case that has already been handled. The version guards against deletion using a stale screen.

Original text and later replies

The original submission is in feedback.feedback. Subsequent messages are in chat. The detail response joins the case context and loads messages, and CSM can also receive audit history. Not every role receives the same detail fields or controls.

Check yourself: Why can search show zero after a case was saved?

Answer: A type, status, topic, page or permission filter may exclude it. Check filters and the authenticated role before assuming an insert failed.

Assignment: CSM sends work to the right person

The business workflow

All customer enquiries and complaints first enter CSM triage. CSM reads type/topic/order context and either handles the request or assigns an active operational user. Payments may go to a manager; garment problems to staff; pickup timing to a rider. Topic helps a person decide; it does not itself guarantee automatic routing.

The endpoint and DTO

PATCH /api/support/cases/{id} invokes SupportService.handle with CaseUpdate: status, priority, assigneeId, note and version. The service checks operational access and retrieves the protected case. Feedback is rejected because it does not follow assignment/resolution handling.

Permission and integrity checks

Only coordinator() roles can change the assignee. Other allowed operational users cannot reassign to someone else. The proposed assignee must exist, be active and have an eligible operational role. A case progressing beyond New must have an assignee. Resolved or Closed handling requires a note and an allowed transition.

```
UPDATE feedback
SET caseStatus=?, priority=?, assigneeID=?,
    version=version+1, updatedAt=SYSDATETIME()
WHERE feedbackID=? AND version=? AND deleted=0
```

The values come from CaseUpdate. If no row matches the current version, changed(...) rejects the stale write. Audit records the previous/new status, priority, assignee and note. The customer receives an update notification; a changed assignee receives an assignment notification linking to the case.

Check yourself: Can a manager reassign a support case merely because they are a manager?

Answer: No. The coordinator role is CSM or CUSTOMER_SERVICE_MANAGER. Seeing all cases and coordinating assignment are separate capabilities.

Read the normal status rules

New can move to Assigned or In Review. Assigned can move to In Review. In Review can move to Resolved. Resolved can move to Closed or Reopened. Closed can move to Reopened. Reopened can move to Assigned or In Review. TRANSITIONS describes these changes; allowed() also permits an unchanged status. Customer satisfaction has its own guarded closure path, explained on page 11.

Replies: from a chat button to saved messages

The entry point

cases.js sends MessageInput to POST /api/support/cases/{id}/messages. SupportController.message delegates to SupportService.message. @NotBlank and @Size(max=250) validate the text; the service trims it before writing.

Access and state before writing

one(actor,id) checks whether the actor can access that case. A Closed enquiry/complaint rejects new messages. Feedback rejects customer follow-up, rejects replies by non-CSM operational users, and rejects a second operational response. These are server checks even if the frontend also hides the reply box.

```
UPDATE feedback SET updatedAt=SYSDATETIME(),
    version=version+1
WHERE feedbackID=? AND deleted=0 AND version=? ...

INSERT INTO chat(message,userID,feedbackID) VALUES (?,?,?)
```

The first write updates the version before inserting the chat row. This coordinates messages with concurrent closure or deletion. The chat row identifies the message author and case. @Transactional means a later failure should roll the earlier write back.

Who is notified

For a customer reply, the assignee is notified; an unassigned request follows the code's role-notification path. For an operational reply, the owning customer is notified. An audit entry records that communication occurred. The service returns updated detail for rendering.

What refresh means

The reply is stored in SQL Server; it is not merely a JavaScript bubble. Other role views must fetch the updated detail. Support polling provides periodic refresh, while a successful own action can immediately redraw from its response.

Check yourself: Does returning JSON automatically update the visible chat?

Answer: No. The browser handler must receive it and render the updated conversation.

Customer Yes / No and case closure

When confirmation is meaningful

The customer must be the owning customer, the record must be Question or Complaint, and it must not already be Closed. The latest chat author must be operational rather than the customer. A customer cannot repeatedly confirm without another support reply between confirmations.

```
POST /api/support/cases/{id}/satisfaction
{ "satisfied": true, "version": 4 }
```

SatisfactionInput requires a Boolean choice and a nonnegative version. The example version is illustrative: the frontend must use the current case version, not a hard-coded number. SupportService.satisfaction is the business method.

```
String status = input.satisfied()
    ? "Closed"
    : "Resolved".equals(item.get("status"))
        ? "Reopened" : "In Review";
```

Read ? : as an if/else expression. If satisfied is true, choose Closed. Otherwise, if the previous status was Resolved choose Reopened; otherwise choose In Review. This customer-confirmation path has its own rules; it is not simply the normal staff TRANSITIONS map.

What is saved

The case status and version update, a standard confirmation message is inserted into chat, an audit entry is added, and CSM plus the assignee are notified. Yes makes the conversation read-only unless an authorized workflow reopens it later. The frontend closes the panel; reopening the record shows the closed result. No permits continued conversation.

Check yourself: Does Yes merely hide the dialog?

Answer: No. It must persist Closed in the backend. Hiding a panel without the database update would not stop direct API replies.

Feedback: a separate journey in the same entry

What the customer sees

The customer chooses Feedback in the Support message-type dropdown. The UI changes to request a rating and experience rather than lead them through assignment and closure. My feedback lets the customer revisit the submission and read the CSM response.

What is shared and what is different

Feedback uses the feedback table and a reply can use chat, but sharing storage does not make the workflows identical. Default support case lists exclude feedback. The CSM feedback page uses cases.js in a feedback-only mode. Feedback does not require an assignee or staff resolution steps.

Operation	Feedback rule
Submit	Customer submits original text with rating 1-5.
Handle/assign	Rejected by SupportService.handle.
Customer follow-up	Rejected by SupportService.message.
Operational reply	Only CSM/coordinator may reply.
Second CSM response	Rejected when an operational chat reply already exists.
Customer Yes/No	Not applicable; satisfaction accepts Question/Complaint only.

How the one-response rule works

message queries chat joined with users and counts authors whose role is not CUSTOMER. If a response already exists, the request conflicts. The case version update also helps serialize simultaneous attempts. The original submission is displayed separately from later reply rows.

Say it in your own words: Feedback is one customer submission and one CSM response. The backend prevents it becoming a conversation, even if someone manually sends another API request.

Security, validation, transactions and concurrency

Four different checks

Authentication asks who you are: login/session. Authorization asks what you may do: URL roles and service access. Field validation asks whether the input has the required shape. Business validation asks whether the operation makes sense now, such as whether the order belongs to you or the case can be closed.

Version example

Two people open case version 3. CSM saves assignment: the row becomes version 4. Another browser submits version 3. WHERE version=3 matches zero rows, so the backend rejects the stale operation instead of overwriting the newer assignment. The user must refresh and reconsider the latest data. This is optimistic concurrency control.

Transaction example

Suppose assignment updates the case but its audit insert fails. With a transaction, the operation should roll back rather than leave a successful-looking assignment without the required related writes. Transactions protect grouped database changes; they do not guarantee external email delivery.

Parameterised SQL example

SELECT ... WHERE orderID=? AND userID=? passes the ID values as arguments. They are data, not extra SQL commands. Fixed SQL structure and permitted filter values matter too: using placeholders for values does not make arbitrary user-built SQL safe.

Passwords and CSRF

Administration uses PasswordEncoder backed by BCrypt: it stores a one-way hash and login verifies against it; it does not decrypt it. CSRF tokens help protect session-authenticated writes from unwanted cross-site requests. The current rider API configuration has an exception, so do not claim every endpoint has identical CSRF protection.

Check yourself: Why do DTO checks not prove security?

Answer: A perfectly formatted request can still target another customer or ask for an unauthorized assignment.

Administration: catalogue, prices and accounts

Catalogue flow

Manager service_catalog.html uses handlers in support/common.js and SupportController administration routes. AdministrationService validates and persists service definitions and item/service prices. ServiceInput includes name, description, turnaroundHours, active and version. PriceInput includes itemId and a nonnegative two-decimal price.

Service versus price

A service describes an offering such as Wash and Fold. A servicePricing row associates an item with a service and a price. Changing a service description is different from changing the price for a shirt. Existing order line totals are historical records; do not claim a catalogue update automatically rewrites past orders.

Operational account flow

staff_accounts.html calls administration endpoints. StaffInput carries names, email, phone, role, password, active and version. Allowed role input is RIDER, STAFF, CSM or MANAGER. Names/email/phone are validated; duplicate data and permissions are checked by the service. Creation needs a suitable password; account updates have their own rules.

```
private final PasswordEncoder passwords;

// Representative use within account creation:
String encoded = passwords.encode(input.password());
```

The actual field and call appear in AdministrationService. SecurityConfig supplies the encoder implementation. The database stores the encoded result. Do not show plaintext passwords from logs or suggest that the stored hash can be decoded.

Demonstrating CRUD safely

Create a disposable catalogue/account record, read it after refresh, make an allowed update and show deactivation/soft deletion where supported. Try an invalid price or duplicate email to show a clear rejection. Avoid deleting required records or real history just to demonstrate CRUD.

Reports: explain every number accurately

The flow

The reports page sends selected dates/service to /api/support/reports. SupportController.report delegates to ReportService. ReportService checks role and filter validity, queries SQL Server and returns structured results. reports.js renders totals and graphs and builds CSV from that returned data.

What the date period means

The cohort selects orders by earliest recorded status activity. It is not a generic filter on every payment timestamp. For example, an order first recorded on 8 October with payment recorded on 10 October belongs to the 8 October order cohort; its recorded payment can contribute to that cohort's total.

What the service filter means

The filter selects orders containing that service. Because the cohort is orders, do not assume a mixed-service order's total is only the selected service's line value. Read the exact aggregate query to explain the number.

Metric	Meaning
Total orders	Number of selected order records.
Order value	Sum of stored order-line totals for selected orders.
Recorded payments	Stored payment amounts for selected orders; not proof of bank settlement or audited net revenue.
Delivered/completed	Selected orders in the implementation's completion status set.
Support counts/alerts	Operational support aggregates have their own scope; do not assume every number shares the selected order period.

Charts, CSV and live data

Graphs visualize returned aggregates; they do not create another database source. CSV is assembled by the browser. Dashboards poll persisted records: the manager view refreshes every 30 seconds, support views every 15. This is periodic updating, not WebSockets or instantaneous push.

Check yourself: Should you multiply linePrice by quantity again?

Answer: No. In this project linePrice is already the line total used by reporting.

Notifications and rider assignment integration

Notifications are persisted messages

NotificationService writes notifications for a user or a set of roles. A notification has a recipient, category, text and a link such as cases.html?caseId=197. The link identifies a case, not necessarily the order with that same number. The bell in global-pre.js fetches the recipient's inbox and tracks unread/read state.

Examples in your support code

Creation notifies CSM. Assignment notifies a new assignee and updates the customer. Operational reply notifies the customer. Customer reply notifies the assignee or the configured unassigned role group. Customer confirmation notifies CSM and the assignee. Avoid claiming every event notifies every role.

Rider assignment is another integration

Manager assign_riders.html uses manager/rider-assignments.js. RiderAssignmentController.java contains both the controller and a package-private RiderAssignmentService in the same source file. Assignment checks stage and eligible rider, locks relevant rows and updates delivery/order records. UPDLOCK/HOLDLOCK serialize competing assignment operations. Later transport actions belong to RiderService/RiderRepository.

Why staff may see zero after payment approval

Verified payment releases the order toward pickup. A rider must collect and hand the order over at the shop. Only then does the facility queue have an In Shop order to receive. Staff work is a shared facility queue rather than automatic individual staff assignment. A manager's all-time total counts orders at earlier stages too.

Say it in your own words: A notification tells a person there is work; the permission-scoped API and dashboard determine which work they can actually open.

Your pages, scripts and backend map

Page under static/html/	Script / purpose
customer/feedback.html	support/cases.js: Support entry, own cases and My feedback.
support/cases.html; admin/customer-service-manager/complaints.html	support/cases.js: triage, assigned cases and conversation.
support/feedback.html	cases.js feedback-only mode: CSM one-response feedback.
CSM dashboard; support/dashboard.html	support/overview.js: case summaries and recent work.
CSM orders.html	support/orders.js: order lookup and histories.
admin/manager/dashboard.html	manager/dashboard.js: live business totals and alerts.
admin/owner/reports.html	support/reports.js: filters, charts and CSV.
admin/manager/service_catalog.html; staff_accounts.html	support/common.js: administration handlers.
admin/manager/assign_riders.html	manager/rider-assignments.js: transport assignment.

How to find any button

Open the HTML, inspect its script tags, and find the button ID/form or event listener in that script. Read the requested API URL. Search the Java source for the controller mapping, then follow the service call and SQL write. This is more reliable than guessing from the page title.

Shared frontend files

HTML supplies elements; CSS supplies layout; JavaScript controls interaction and fetches data. global-pre.js supplies role navigation and the notification bell. support/common.js has shared API helpers as well as administration handlers. The script tags and data-page mode are authoritative when one controller serves several pages.

Check yourself: Where would you fix a missing reply bubble?

Answer: First examine the cases.js request/rendering and response; then check the backend detail/message data. Do not assume the insert failed just because rendering is wrong.

OOP basics: class, object and encapsulation

Class and object

A class defines fields and methods. An object is a runtime instance of that class. SupportService is a class; the Spring-managed service instance is an object. Different classes have responsibilities and collaborate. A database row and a Java object are related concepts, but not automatically the same thing.

```
private final SupportRepository repo;
private final SupportAccess access;
private final NotificationService notifications;
```

Encapsulation in your service

These fields are private, so callers use public methods such as create or handle rather than directly replacing collaborators. The public workflow method organizes the permission checks and writes. Encapsulation is about controlled access to internal data/behaviour, not encryption.

Abstraction in your repository

SupportService calls repo.update(sql,args). SupportRepository delegates to JdbcTemplate and returns the affected-row count. The caller does not need to repeat JDBC invocation details. This abstraction is partial: the service still knows SQL and table structure. BillingRepository shows a stronger domain-oriented interface boundary.

Objects collaborating

Your constructor receives repository, access and notification objects. Spring dependency injection supplies them. This is a has-a/dependency relationship: the service uses those objects. Shared injected beans are not evidence of strict UML composition where their lifetimes belong exclusively to the service.

Check yourself: Does final mean the database can never change?

Answer: No. It prevents reassigning that Java field reference. The referenced repository can still execute updates.

Say it in your own words: My service encapsulates dependencies and exposes workflow methods. It collaborates with access, repository and notification objects through constructor injection.

OOP: overloading, overriding and polymorphism

Overloading: same name, different parameters

SupportService has two cases(...) method signatures, one without topic and one with topic. The shorter method delegates to the fuller one with a default/null topic. Java chooses the matching signature at compile time. CaseInput similarly provides an extra constructor that defaults the topic to General.

```
// Illustrative shortened signatures:
cases(Actor actor, String search, ..., int page)
cases(Actor actor, String search, ..., int page, String topic)
```

Different names such as staff() and manager() are not overloading. A comment that says overloading is not enough: show the actual same-name signatures with different parameter lists. Some broad existing comments mislabel role helper methods; use the declarations as the evidence.

Overriding: implement the same contract

PercentageDiscountStrategy implements DiscountStrategy and defines calculate(...) with @Override. The fixed-amount class implements the same method differently. The method supplies the interface contract; it is not an overload merely because another class also has calculate.

Runtime polymorphism in your administration

AdministrationService calls a PasswordEncoder reference. SecurityConfig supplies BCryptPasswordEncoder. The caller depends on the interface; the runtime object performs encoding. This is an actual example within your component, even though the implementation comes from Spring Security.

Inheritance: be precise

Repository interfaces such as UserRepository extend JpaRepository and inherit its operations; exception subclasses inherit standard exception behaviour. Your support workflow does not have a Customer extends User / Manager extends User hierarchy. Roles are values checked by Actor helpers. Interface implementation and class inheritance are related OOP mechanisms, but name the actual syntax.

Check yourself: How do you tell overload from override?

Answer: Overload: same name with different parameter lists. Override: an implementation supplies an inherited/interface method with the corresponding signature.

Strategy pattern: understand promotion discounts

The problem

A percentage promotion and a fixed-amount promotion need different formulas. Strategy places those interchangeable calculations behind one contract, so the calling service does not select the formula using an if/else for each type.

Eligible amount	Promotion	Discount
LKR 2,000	10 percent	2,000 x 10 / 100 = LKR 200
LKR 2,000	LKR 300 fixed	LKR 300
LKR 2,000	LKR 3,000 fixed	Capped by service to LKR 2,000

```
interface DiscountStrategy {
    BigDecimal calculate(BigDecimal subtotal,
                         BigDecimal discountValue);
}
```

The contract specifies inputs and result, not the formula. PercentageDiscountStrategy multiplies and divides by 100, rounded to two places with HALF_UP. FixedAmountDiscountStrategy returns discountValue. BigDecimal is used for decimal money calculations.

```
PERCENTAGE(new PercentageDiscountStrategy()),
FIXED_AMOUNT(new FixedAmountDiscountStrategy());

DiscountStrategy strategy =
    promotion.getDiscountType().strategy();
BigDecimal discountAmount =
    strategy.calculate(subtotal, promotion.getDiscountValue());
```

DiscountType holds the appropriate calculator object. PromotionService asks the type for that object, then calls calculate. The actual object determines which implementation executes. The service separately validates the promotion and caps the discount; the strategy only performs its formula.

Say it in your own words: The interface is the contract, the two classes are concrete strategies, and PromotionService uses the selected strategy polymorphically. This is the promotion module, outside my major component.

Strategy for processing and what counts as two

The same pattern in a different feature

ProcessingRoute defines route behaviour. WashRoute, DryCleanRoute, ShoeCleanRoute and IroningRoute implement different cleaning paths. ProcessingTransitions.routeFor(serviceNames) selects the route; ProcessingService asks it about the allowed next step or rework. The route selector is not by itself a GoF Factory Method pattern.

A physical example

A wash order can require Washing -> Drying -> Ironing -> Quality Inspection. A dry-clean order follows Dry-Clean -> Ironing -> Quality Inspection. The common contract lets the processing service use a route without duplicating each route's decisions in a long chain of conditional statements.

Benefit and extension

A new route can be implemented behind the same contract and connected to selection. Adding a discount likewise requires its concrete strategy and type wiring, plus relevant validation/UI/storage updates. Do not promise that only one new class is ever needed: the surrounding application must know about the new option.

Two uses are not two different patterns

Processing Strategy and Discount Strategy demonstrate one pattern applied twice. For two distinct examples, present Strategy plus Repository. Repository is an enterprise data-access pattern; layered/MVC-style separation is architectural. If the module specifically expects two GoF patterns, confirm that classification with the lecturer rather than relabel code.

Claims to avoid

A status enum is not automatically State. Polling or SQL triggers are not a custom Java Observer implementation. Spring's default singleton bean scope is not evidence of a handwritten Singleton pattern. An annotation or an explanatory comment alone does not prove an implemented pattern.

Check yourself: What should you show for Strategy?

Answer: The interface, concrete implementations, selection and real caller. Showing only an enum or class name is insufficient.

Repository pattern: your second example

The purpose

A repository provides operations for retrieving and persisting data. It gives calling code a place to request records instead of duplicating database execution throughout controllers. Its usefulness comes from the boundary and operations, not merely the word Repository in the filename.

```
public interface BillingRepository {
    Optional<Integer> findOrderUserID(Integer orderID);
    List<BillingLine> findOrderLines(Integer orderID);
    BigDecimal findAppliedDiscountAmount(Integer orderID);
}

class BillingJdbcRepository implements BillingRepository {
    // Implements those operations using JdbcTemplate and SQL.
}
```

BillingService receives BillingRepository in its constructor, then calls these operations. The interface describes what billing needs. BillingJdbcRepository contains how SQL Server/JDBC supplies it. Tests can substitute a controlled repository implementation/mock to exercise billing without repeating real SQL execution.

In your component

SupportRepository wraps query, update, count, insert and audit using a private JdbcTemplate. SupportService, AdministrationService and ReportService use it. CASE_SELECT joins the case with customer and optional assignee data. A returned Map stores named columns; this is explicit JDBC rather than a JPA support entity repository.

The honest limitation

SupportService assembles many SQL statements itself. Therefore support is not a fully database-independent domain layer, and changing schema can affect services. Use BillingRepository/BillingJdbcRepository as the clearer interface/implementation example, then show your SupportRepository as the component-specific persistence helper.

Say it in your own words: Repository separates persistence operations from HTTP handling and supports testing. My support repository centralizes JDBC execution, although its services still contain SQL.

Changing code and diagnosing a failure

Requested change	Start with
Case text/layout	HTML and CSS; cases.js for dynamic labels/controls.
New payload field	SupportRequests + frontend JSON + service; schema migration if saved.
Who can see/assign	SupportAccess + service guards; frontend role controls too.
Closing/chat rules	SupportService.satisfaction/message and cases.js.
Status transitions	SupportService.TRANSITIONS/allowed/handle; confirmation has its own path.
Report totals	ReportService SQL; reports.js for visual presentation.
Price/account validation	AdministrationService and relevant DTO/handler.
Notification recipient	NotificationService caller; global-pre.js for bell rendering.
Rider assignment	RiderAssignmentController and same-file service.

A reply did not appear: diagnose in order

Check whether the click handler ran and an HTTP request was sent. Check its response status and error body. Check that the actor and case ID are right. Check whether chat contains the row. Check that detail returns it and the frontend renders it. Refresh the recipient view. A missing visual bubble can be a read/render problem even when a write succeeded.

A save conflicts

409 can mean stale version, invalid transition, Closed chat or a second feedback reply. Read the message and refresh relevant state. Removing the version condition to silence an error risks lost updates; explain and repair the cause instead.

What an examiner may ask you to modify

Explain the intended layer before editing. A CSS change is different from a permission rule. Add tests for changed business behavior, compile, and demonstrate the affected workflow. Never guess that a successful notification alone proves all database changes were correct.

A ready-to-run support demonstration

Customer	Password / completed order
vipusha@gmail.com	Vipusha1234 / #197 / LKR 660
moneesha@gmail.com	moneesha1234 / #198 / LKR 1,500

Both accounts logged in successfully and their Delivered, VERIFIED-payment orders were available through the support API. These orders were created directly in SQL with lifecycle records; they do not prove UI order-processing actions were exercised. Credentials below are local demonstration credentials.

Role	Login
CSM	cathy@csm.com / Cathy1234
Manager	maya@manager.com / Maya1234
Staff	sam@staff.com / Sam1234
Rider	ravi@rider.com / Ravi1234
Owner	oliver@owner.com / Oliver1234

Enquiry demonstration

Vipusha submits Question -> Payments & billing for order #197: Could you explain how my charge was calculated? CSM opens it, assigns Maya and shows the audit. Manager opens the notification/assigned case and explains the price. Customer chooses No and asks a follow-up; manager replies again. Customer chooses Yes, then reopens the record to show read-only closure.

Complaint and feedback

Moneesha submits Complaint -> Laundry & items for #198. CSM assigns Sam; staff replies and the customer confirms. Use Feedback from the same dropdown for a separate experience/rating. CSM replies once; the customer sees the response with no chat input or Yes/No handling.

Complete your component

Show catalogue or operational-account CRUD with invalid input, then a report with dates/service, charts and CSV. Use separate browser profiles for roles to avoid one login replacing another. Rehearse and refresh before the presentation.

Order journey and honest readiness limits

Stage	Responsible action
Booking/payment	Customer saves order and submits payment; authorized management verifies.
Awaiting Pickup	Manager assigns pickup rider; payment approval alone does not send work to washing.
Transport to shop	Rider collects and records shop handover.
In Shop / receiving	Staff receive/count items into the shared facility queue.
Cleaning	Staff follow the selected route; invalid jumps are rejected.
Quality / readiness	Quality inspection, permitted rework if needed, packing and ready action.
Delivery	Manager assigns delivery rider; rider completes transport; order becomes Delivered.

Your latest evidence

The latest recorded full suite passed 356 tests with no failures/errors. A fresh frontend audit checked 55 HTML pages and 28 JavaScript files with no broken literal local links, syntax errors or missing required controls. Role/login/API checks and database integrations provide useful evidence, but they do not certify every button visually or replace a timed rehearsal.

Remaining practical limits

SMTP is not configured, so actual reset-email delivery is unverified. No production payment gateway settlement, live GPS or performance/uptime guarantee is established by a local demo. Reports are stored-record summaries and polling refreshes. Latest local changes still need to be included in the submitted source baseline.

Excellent preparation is more than code

The specification gives 20 minutes for the team demo and 10 minutes Q&A.; Every member should present their function and hand over smoothly. The final report must be PDF, at most 25 pages, with current screenshots, completed personal reflections and IEEE references. The available report draft still has placeholders and outdated statements; this pack is not a replacement.

Examiner questions: answer, then explain why

What happens when the customer presses Submit?

JavaScript sends JSON to the API. The controller binds it, the service checks input/ownership, and the repository writes the case. Audit and CSM notifications follow; returned detail redraws the screen.

Why not put all code in the controller?

Separating HTTP, business decisions and persistence makes responsibilities clearer and permits tests without driving every case through a browser.

What is a DTO?

An API data shape. CaseInput describes the submitted fields and validation; it is not automatically a database entity.

How do you prevent unauthorized access?

Authenticated identity, URL permissions and service-level ownership/assignee checks. Hidden UI controls do not protect direct API calls.

Where does the case status change?

handle updates feedback.caseStatus for staff handling; satisfaction updates it for customer Yes/No. orders.statusID is a separate physical order lifecycle.

Why use a version?

To reject updates based on old data. The UPDATE matches both ID and version, increments it, and fails when another operation already changed the row.

Can feedback keep chatting?

No. Customer follow-up and a second operational reply are rejected. Only CSM gives the one response.

Which OOP concepts are in your component?

Private fields: encapsulation; repository helpers: abstraction; constructor collaborators: dependency injection/has-a; cases overloads; PasswordEncoder interface polymorphism.

Practice answers and a final self-test

What are your two pattern examples?

Strategy for interchangeable route/discount algorithms; Repository for persistence operations. Explain where they are called. The two Strategy applications are not two distinct patterns.

What is live about your reports?

They query persisted database records and the browser refreshes them periodically. Date selection is an order cohort, not necessarily the dates of all contributing payments.

What did you contribute?

Describe the reporting, administration and support work you understand, with actual features, files, tests and design decisions. Ownership does not mean exclusive authorship of every shared file.

What would you improve?

Mention specific limits supported by evidence: SMTP delivery, browser acceptance coverage, deployment/monitoring or clearer persistence separation. Do not invent measured outcomes.

Explain these without looking

1. Trace creation through frontend, controller, service, repository and database. 2. Explain why only CSM reassigns. 3. Trace a reply and notification. 4. Explain Yes/No and the latest-reply rule. 5. Explain one-time feedback. 6. Explain version conflicts with two browsers. 7. Explain a report date filter with an example. 8. Show one OOP concept and its exact code.

How to study when you are starting from zero

First read pages 2-6 and explain the layers aloud. Then perform the page 24 demonstration while following pages 7-17. Open the OOP/pattern files and identify the real declarations/calls. Answer pages 26-27 without reading; revisit weak answers. Finally rehearse with a teammate asking follow-up questions. A memorised sentence is a starting point, not proof of understanding.

Be candid during questions

If uncertain, identify the method and read the decision before answering. Say what you know and distinguish it from what you would check. Do not claim tests, authorship, patterns or outcomes you cannot defend. The course expects you to understand and explain your work even when learning tools helped you.

Source locator: open the exact implementation

Paths below are relative to src/main/java/_6/Y2/S1/MTR/_6/LaundryLink/. Line numbers are generated from the current local source. Formatting changes can shift them; search the method name if needed.

File and line	What to show
support/SupportController.java:126	Case creation HTTP entry
support/SupportService.java:300	Validate and save case
support/SupportService.java:418	Assignment/status writes
support/SupportService.java:532	Reply/feedback rules
support/SupportService.java:620	Customer Yes/No
support/SupportRepository.java:19	JDBC operations and CASE_SELECT
support/SupportAccess.java:41	Role capabilities
support/dto/SupportRequests.java:21	DTO constraints
support/AdministrationService.java:39	Catalogue/accounts business rules
support/ReportService.java:29	Cohort/aggregate SQL
support/RiderAssignmentController.java:84	Same-file assignment service
promotion/DiscountStrategy.java:18	Strategy contract
promotion/DiscountType.java:9	Type/strategy mapping
promotion/PromotionService.java:228	Runtime strategy call
processing/ProcessingRoute.java:25	Processing Strategy contract
processing/ProcessingTransitions.java:76	Route selection
billing/BillingRepository.java:17	Repository contract
billing/BillingJdbcRepository.java:29	SQL implementation
billing/BillingService.java:40	Repository injection

Further local references: Documentation/Viva-Code-Guide.md, Viva-Backend-File-Index.md, Viva-Page-Index.md and Viva-Demonstration-Checklist.md. Search OOP comments, but verify every label against the actual declaration. Specification: SE2030_Group Project Specification.pdf, final rubric pages 18-23; presentation/report instructions pages 5-8.